第二章作业

1

给定一个带头结点的单链表 L，设计一个算法将链表中的所有元素原地逆置，要求只使用 $O(1)$ 的额外空间。

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        pre = None
        cur = head
        while cur:
            nxt = cur.next
            cur.next = pre
            pre = cur
            cur = nxt
        return pre
```

2

给定一个**不带头结点**的单链表 L，其中 head 指向第一个数据结点，设计算法找到其中间节点。若节点数为偶数，规定返回两个中间节点中的**后一个**。要求只遍历链表一次。

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def find(self, head: Optional[ListNode]) -> Optional[ListNode]:
        quick, slow = head, head
        while quick and quick.next:
            quick = quick.next.next
            slow = slow.next
        return slow
```

3

给定两个**无环单链表** L_1, L_2，请设计算法判断它们是否相交；若相交，找出它们的第一个公共节点。要求 $O(1)$ 额外空间。

```python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) ->
Optional[ListNode]:
        p1, p2 = headA, headB
        l1, l2 = 0, 0
        while p1:
            p1 = p1.next
            l1 += 1
        while p2:
            p2 = p2.next
            l2 += 1
        p1, p2 = headA, headB
        for _ in range(l1 - l2):
            p1 = p1.next
        for _ in range(l2 - l1):
            p2 = p2.next
        while p1 != p2:
            p1 = p1.next
            p2 = p2.next
        return p1
```